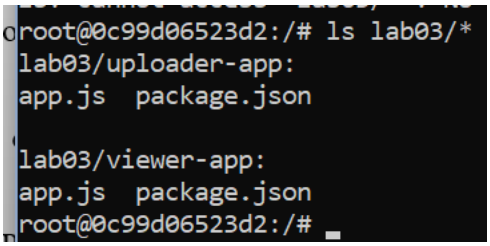
Screenshot of your local lab03/uploader-app/ and lab03/viewer-app/ folders showing app.js and package.json in each.



Screenshot of the Uploader page and a successful upload confirmation.

Screenshot of attempting to upload a .txt file larger than 1MB, showing the “file exceeds the 1MB limit” error.

Screenshot of attempting to upload a non- .txt file (e.g., a .jpg or .pdf), showing the “only .txt files are allowed” error.

Screenshot of the Viewer page displaying the uploaded file’s content.

Screenshot of the Viewer page after uploading a second, different file and clicking Refresh, showing the updated content.

Screenshot of aws ec2 describe-launch-templates showing both templates.

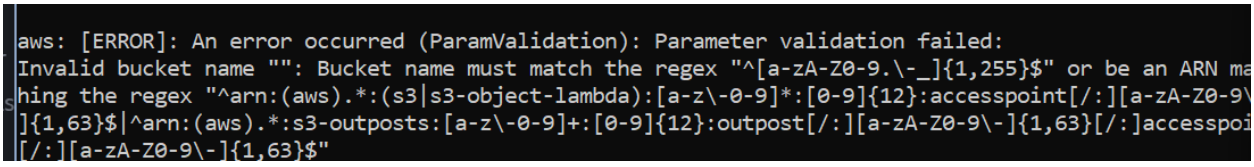
Screenshot of aws ec2 describe-launch-template-versions --launch-template-name <uploader-lt-name> after rerunning the create script twice, showing more than one version.

aws iam get-role-policy output (or screenshot) for both roles, showing each has only its one intended action on the exact shared.txt key.

Screenshot of create_app_stack.sh output showing both instances running with public IPs.

Screenshot of delete_app_stack.sh completing with no errors.

INSTEAD, an image of the error I cannot fix:



One paragraph explaining why the uploader and viewer use separate IAM roles instead of one shared role with both permissions.

The uploader and viewer use separate IAM roles instead of one shared role because the source of the information doesn’t necessarily have the same clearance for information as a viewer

would. This is important for logical security purposes. It can be used with the principle of least privilege or with general role-based access control.

One or two sentences explaining why embedding code directly into user-data means the instance never needs a Git or S3 credential just to fetch its own code.

Embedding code into user-data lets it be used at launch instead of it needing to be fetched from Git or S3. This is because user-data includes its information in the instance startup instructions.